



LibHEIF Introduction

Open source library for HEIF images and sequences

Dirk Farin – Algorithmic Research e.K.

27 January 2026

OGC Code Sprint



What is LibHEIF

- Open-Source implementation of HEIF standard.
 - HEIC / AVIF / GIMI
- Stable C-language API
 - Easy integration into other languages
 - Can be compiled to JavaScript/WebAssembly
- In development since 2017 (first HEIF standard).
- Security limits and fuzz-tested to prevent DoS attacks.
- Widely used in image/photo processing software.
- <https://libheif.org>



Compression Formats and Plugins

- HEIF supports many different compression formats
- LibHEIF supports each format with codec plugins
 - User can choose encoder/decoder
- Plugins can either be
 - Compiled directly into LibHEIF → easier handling
 - Compiled in separate plugin files and loaded at runtime → more flexible packaging, smaller footprint
- LibHEIF also includes implementation of ISO23001-17 codec (uncompressed image and video)
 - Only built-in, no plugin

Format	Decoders	Encoders
HEIC	libde265, ffmpeg	x265, kvazaar
AVIF	libaom, dav1d	libaom, rav1e, svt-av1
VVC	vvdec	vvenc, uvg266
AVC	openh264, ffmpeg	x264
JPEG	libjpeg(-turbo)	libjpeg(-turbo)
JPEG2000	OpenJPEG	OpenJPEG
HTJ2K	OpenJPEG	OpenJPH
uncompressed	built-in	built-in

From Where Can I Get It ?

- LibHEIF is available for
 - Most Linux distributions.
Often included in main package repository.
Debian/Ubuntu:
 - apt install libheif # library
 - apt install libheif-dev # C development headers
 - apt install libheif-examples # command-line tools
 - + more codec plugins in separate packages
 - Windows:
 - vcpkg
 - MSYS2 / cygwin
 - MacOS:
 - Homebrew / MacPorts

libheif packaging status					
Alpine Linux 3.20	1.17.6	Fedora 42	1.19.8	Parrot	1.19.8
Alpine Linux 3.21	1.19.5	Fedora 43	1.20.2	PCLinuxOS	1.15.1
Alpine Linux 3.22	1.19.8	Fedora Rawhide	1.21.2	Pisi Linux	1.21.0
Alpine Linux 3.23	1.20.2	FreeBSD Ports	1.21.2	pkgsrc-2025Q3	1.20.2
Alpine Linux Edge	1.21.2	Gentoo	1.20.2	pkgsrc-2025Q4	1.20.2
ALT Linux p9	1.6.2	GNU Guix	1.19.7	pkgsrc current	1.21.2
ALT Linux p10	1.19.5	HaikuPorts master	1.19.8	PLD Linux	1.20.2
ALT Linux p11	1.19.5	Homebrew	1.21.2	PureOS amber	1.3.2
ALT Sisyphus	1.21.2	Kali Linux Rolling	1.21.2	PureOS byzantium	1.11.0
AOSC	1.20.2	KaOS	1.21.2	PureOS landing	1.19.8
Apertis v2025	1.15.1	LiGurOS stable	1.20.1	Raspbian Oldstable	1.15.1
Apertis v2026 Development	1.19.8	LiGurOS develop	1.20.2	Raspbian Stable	1.19.8
Arch Linux	1.21.2	MacPorts	1.20.2	Raspbian Testing	1.20.2
ArchPOWER powerpc	1.20.2	Mageia cauldron	1.20.2	Ravenports	1.21.2
ArchPOWER powerpc64le	1.20.2	Manjaro Stable	1.21.1	Rosa 2021.1	1.12.0
ArchPOWER riscv64	1.19.7	Manjaro Testing	1.21.2	Rosa 13	1.19.8
AUR	1.21.2	Manjaro Unstable	1.21.2	RPM Fusion EL 8	1.7.0
Artix	1.21.2	MidnightBSD mports	1.19.8	Side Linux	1.20.2
Chimera Linux	1.20.2	MSYS2 clang64	1.21.2	SlackBuilds	1.20.2
Chromebrew	1.20.2	MSYS2 clangarm64	1.21.2	Slitaz Next	1.3.2
ConanCenter	1.20.1	MSYS2 mingw	1.21.2	Solus	1.21.2
CRUX 3.8	1.20.2	MSYS2 ucrt64	1.21.2	Spack	1.12.0
Cygwin	1.12.0	MX Linux MX-21 Testing	1.14.0	stal/IX	1.20.2
Debian 11	1.11.0	MX Linux MX-23 Testing	1.17.6	stal/IX dev	1.20.2
Debian 12	1.15.1	nixpkgs stable 25.11	1.20.2	T2 SDE	1.21.2
Debian 12 Backports	1.19.7	nixpkgs unstable	1.20.2	Termux	1.21.2
Debian 13	1.19.8	OpenBSD Ports	1.21.2	Trisquel 11.0	1.12.0
Debian 14	1.21.2	OpenIndiana packages	1.20.2	Ubuntu 18.04	1.1.0
Debian Unstable	1.21.2	openmamba	1.20.2	Ubuntu 20.04	1.6.1
deepin 20	1.3.2	OpenMandriva 6.0	1.19.7	Ubuntu 22.04	1.12.0
deepin 23	1.18.1	OpenMandriva Rolling	1.20.2	Ubuntu 24.04	1.17.6
Devuan 4.0	1.11.0	OpenMandriva Cooker	1.21.2	Ubuntu 25.10	1.20.2
Devuan Unstable	1.21.2	openSUSE Tumbleweed	1.21.2	Ubuntu 26.04	1.21.2
Endless OS master	1.15.1	openSUSE multimedia:libs Tumbleweed	1.21.2	Vcpkg	1.21.2
EPEL 9	1.16.1	PackMan openSUSE Tumbleweed	1.21.2	Void Linux x86_64	1.20.2
EPEL 10	1.17.6	PackMan SLE 15	1.21.2		
Exherbo	1.20.2	Parabola	1.21.2		

Command Line Tools

- LibHEIF is mainly a programming library, but comes with set of command-line tools.
- **heif-dec**: decode HEIF images and sequences to JPEG/PNG, extract metadata
 - Can decode tiled images into individual tile images
- **heif-enc**: encode HEIF images and sequences from JPEG/PNG/TIFF/Y4M
 - Can encode tiled images from set of tiles or cut a large image
- **heif-info**: show information about HEIF content, low-level file structure dump
- **heif-view**: display HEIF image sequences
- **heif-thumbailer**: used by operating systems for image thumbnail generation



Language Bindings

- LibHEIF is written in C++, but public API is pure C.
- Language wrappers exist for
 - JavaScript, WASM, NodeJS - public demo: <https://strukturag.github.io/libheif/>
 - C# / C++ / Go / Python / Rust / Swift / JavaFX

However, most language wrappers target only simple photo loading/saving.
For advanced usage, use the original API.



How Do I Build It From Source ?

1. Get the source code

```
git clone https://github.com/strukturag/libheif.git
```

2. Configure

```
cd libheif  
mkdir build ; cd build  
cmake .. --preset=develop  
ccmake .
```

Optional step to change configuration

3. Build

```
make  
make install
```



LibHEIF Programming Concepts and Base Classes

- Some of the base classes
 - **heif_context**: main class for reading and writing a HEIF file
 - **heif_image_handle**: reference to an image in the HEIF file (not yet decoded)
 - **heif_image**: decoded image (holds pixels and attached metadata)
- **heif_image**
 - Can hold image in several colorspaces (monochrome, RGB, YCbCr).
 - Image data either
 - Planar (R,G,B in separate memory blocks), or
 - Interleaved, RGBA (8 bit) or RRGGBBAA (up to 16 bit)



How Do I Load an Image ?

```
heif_context* ctx = heif_context_alloc();  
heif_context_read_from_file(ctx, input_filename, nullptr);
```

Get new context.
Read HEIF from file.

```
// get a handle to the primary image  
heif_image_handle* handle;  
heif_context_get_primary_image_handle(ctx, &handle);
```

```
// decode the image and convert colorspace to RGB, saved as 24bit interleaved  
heif_image* img;  
heif_decode_image(handle, &img, heif_colorspace_RGB, heif_chroma_interleaved_RGB, nullptr);
```

```
int stride;  
const uint8_t* data = heif_image_get_plane_readonly(img, heif_channel_interleaved, &stride);
```

```
// ... process data as needed ...  
uint8_t pixel = data[x + y * stride];
```

```
// clean up resources  
heif_image_release(img);  
heif_image_handle_release(handle);  
heif_context_free(ctx);
```

Desired output colorspace.
Image will be converted if needed.

Use **heif_colorspace_undefined**,
heif_chroma_undefined to get native
colorspace.



Load from Different Sources

- Images can be loaded from several sources

- File

```
heif_error heif_context_read_from_file(heif_context*, const char* filename,  
                                       const heif_reading_options*);
```

- Memory

```
heif_error heif_context_read_from_memory(heif_context*,  
                                         const void* mem, size_t size,  
                                         const heif_reading_options*);
```

- Reader interface

```
heif_error heif_context_read_from_reader(heif_context*,  
                                         const heif_reader* reader,  
                                         void* userdata,  
                                         const heif_reading_options*);
```



Loading from a **heif_reader** Interface (e.g. Network)

- Provides functions that LibHEIF will use to access data on-demand.
- Can be use to stream large images tile-by-tile over the network.

```
struct heif_reader
{
    int (* read)(void* data,
                 size_t size,
                 void* userdata);

    int (* seek)(int64_t position,
                 void* userdata);
    ...

    // The purpose of this function is that libheif will usually read very small chunks of data with the
    // read() callback above. However, it is inefficient to request such a small chunk of data over a network
    // and the network delay will significantly increase the decoding time.
    // Thus, libheif will call request_range() with a larger block of data that should be preloaded and the
    // subsequent read() calls will work within the requested ranges.

    heif_reader_range_request_result (* request_range)(uint64_t start_pos, uint64_t end_pos, void* userdata);
};
```



Loading Tiled Images – What Kind of Tiled Images Exist ?

HEIF supports three methods to store tiled images:

- “**grid**” – each tile is a separate image item. Another “grid” image combines these into a large image.
Used by most photo cameras. Limited to 65535 tiles. Large overhead because of repeated metadata.
- “**unci**” – native tiling method defined in ISO23001-17 (uncompressed) images.
Only for uncompressed images.
- “**tili**” – tiling format developed in TestBed-20 and included in upcoming HEIF standard.
 - Skipped, undefined tiles
 - Higher-dimensional images (like 3D volumes, multi-spectral images)
 - Optimized for low overhead and network streaming.

LibHEIF transparently reads all of these formats in a unified way.



How Do I Load an Image Tile ?

- First, get
 - the number of tile rows/columns
 - The tile size

```
// --- Get tiling information
```

```
struct heif_image_tiling
{
    uint32_t num_columns;
    uint32_t num_rows;
    uint32_t tile_width;
    uint32_t tile_height;

    uint32_t image_width;
    uint32_t image_height;
    ...
};
```

```
heif_error heif_image_handle_get_image_tiling(const heif_image_handle* handle,
                                             int process_image_transformations,
                                             struct heif_image_tiling* out_tiling);
```

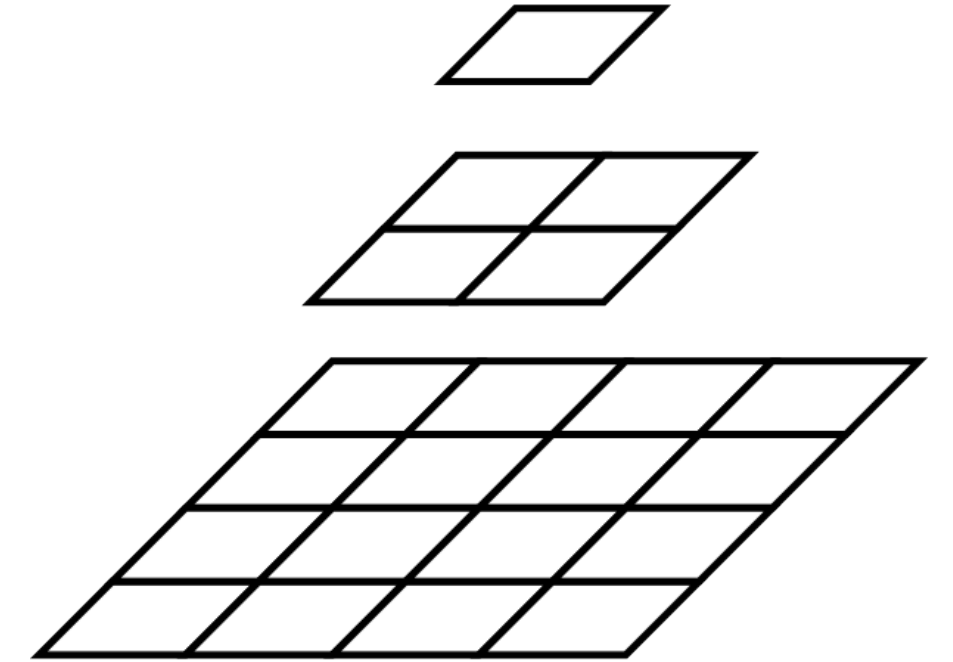
```
// --- Decode single tile image
```

```
// The tile position is given in tile indices, not in pixel coordinates.
heif_error heif_image_handle_decode_image_tile(const heif_image_handle* in_handle,
                                              heif_image** out_img,
                                              enum heif_colorspace colorspace,
                                              enum heif_chroma chroma,
                                              const heif_decoding_options* options,
                                              uint32_t tile_x, uint32_t tile_y);
```

- Then decode a specific tile.
Similar to **heif_decode_image()**.



- First, get the entity-group ID.
- Then get the image ID of a pyramid layer. Decode as usual.
- Pyramids API is still 'experimental'



```
// Get 'pymd' (image pyramid) entity group
```

[illegible]

```
// Get configuration of multi-layer pyramid.
```

```
struct heif_pyramid_layer_info
{
    heif_item_id layer_image_id;    // ID of image at this layer
    ...
    uint32_t tile_rows_in_layer;
    uint32_t tile_columns_in_layer;
};
```

[illegible]

Example: Tiled-Image and Multi-Resolution Image Viewer

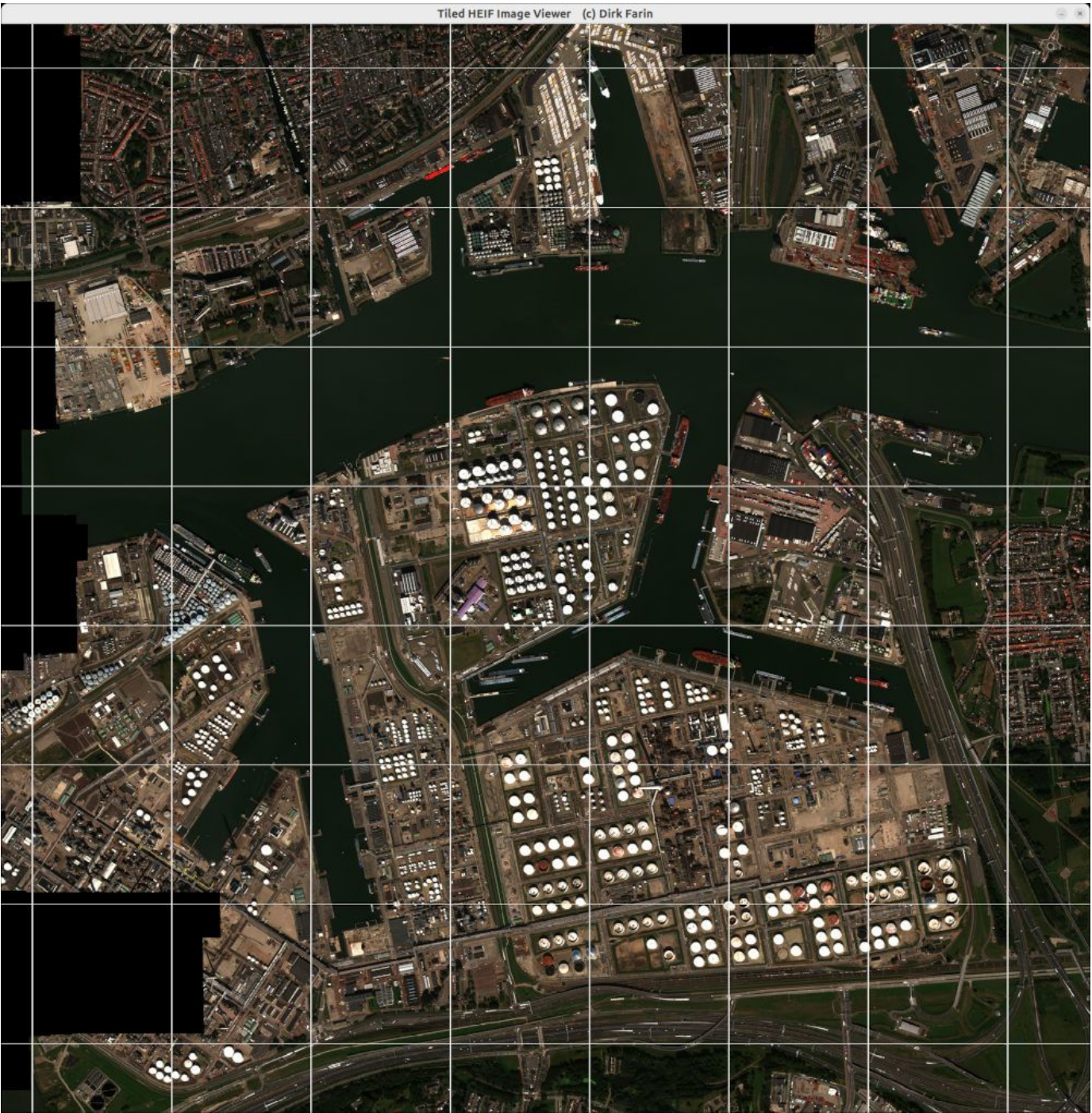
- Example Viewer: <https://github.com/farindk/tiled-image-viewer>

Example Images

Content	Resolution	File Size	Description
Rotterdam	6656x7936	5.3 MB	<code>tili</code> /AV1 pyramid
Rotterdam	6656x7936	22.5 MB	<code>unci</code> /Brotli at highest resolution layer, <code>tili</code> /AV1 at overview layers
Rio	64768x41216	161 MB	<code>tili</code> /AV1 pyramid
Mandelbrot B	65536x65536	304 MB	<code>grid</code> /HEVC pyramid
Mandelbrot C	262144x261120	1.6 GB	<code>tili</code> /AV1 pyramid
M31 Andromeda Galaxy	42208x9870	106.1 MB	<code>grid</code> /HEVC pyramid

Note: final file-format will change when HEIF standard is released.
No geo-metadata in these files.

Video showing how these images were created with **'heif-enc'**:
<https://www.youtube.com/watch?v=7iMEoLLqPyE>



How Do I Write an Image ?

1. Create (empty) context

```
heif_context* ctx = heif_context_alloc();
```

2. Choose encoder

```
// get the default encoder for given compression format
```

```
heif_encoder* encoder;
```

```
heif_context_get_encoder_for_format(ctx, heif_compression_HEVC, &encoder);
```

3. Set encoder parameters

```
// set the encoder parameters
```

```
heif_encoder_set_lossy_quality(encoder, 50);
```

4. Fill image (not shown)

```
heif_image* image;
```

```
heif_image_create(width, height, colorspace, chroma, &image);
```

```
... // code to fill-in the pixels omitted in this example
```

5. Encode image

```
// encode the image
```

```
heif_context_encode_image(ctx, image, encoder, nullptr, nullptr);
```

6. Write to HEIF file

```
// write to file
```

```
heif_context_write_to_file(ctx, "output.heic");
```

```
heif_encoder_release(encoder);
```

```
heif_context_free(ctx);
```



How Do I Write a Tiled Image ?

- Create (empty) 'grid' image.

```
heif_error heif_context_add_grid_image(heif_context* ctx,  
                                     uint32_t image_width,  
                                     uint32_t image_height,  
                                     uint32_t tile_columns,  
                                     uint32_t tile_rows,  
                                     const heif_encoding_options* encoding_options,  
                                     heif_image_handle** out_grid_image_handle);
```

- Add tile images

```
heif_error heif_context_add_image_tile(heif_context* ctx,  
                                     heif_image_handle* tiled_image,  
                                     uint32_t tile_x, uint32_t tile_y,  
                                     const heif_image* image,  
                                     heif_encoder* encoder);
```

- For 'unci', 'tili', there are other functions to create them, but tiles are always added with the same **heif_context_add_image_tile()**.



How Do I Read Metadata Items ?

- Arbitrary metadata can be assigned to an image.
- It is identified by a 4-CC code or a MIME type.

```
// 'type_filter' can be used to get only metadata of specific types, like "Exif".
// If 'type_filter' is NULL, it will return all types of metadata IDs.
int heif_image_handle_get_list_of_metadata_block_IDs(const heif_image_handle* handle,
                                                    const char* type_filter,
                                                    heif_item_id* ids, int count);

// For metadata blocks with MIME type, get the MIME type string.
const char* heif_image_handle_get_metadata_content_type(const heif_image_handle* handle,
                                                        heif_item_id metadata_id);

// --- Get metadata block

size_t heif_image_handle_get_metadata_size(const heif_image_handle* handle,
                                           heif_item_id metadata_id);

heif_error heif_image_handle_get_metadata(const heif_image_handle* handle,
                                          heif_item_id metadata_id,
                                          void* out_data);
```



More HEIF Features to Explore

- Image properties
 - Hold GIMI “ContentIDs”
 - Image transformations (rotation, mirror, crop)
- Region annotations
 - Add area marks, lines, points, ... to images
- Image text descriptions
- Sequences
 - Store short image sequences or MP4 video

